DESIGN AND ANALYSIS OF ALGORITHMS

Assignment-1: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Submitted by: Hemanth P | **Register Number:** 192524297

TASK 1 - DESIGN THE ALGORITHM

This assignment investigates methods for finding a student's unique registration number in a database that may grow from 1,000 to more than 1,000,000 records. Three distinct strategies are compared: linear search, binary search on sorted records, and hash-table lookup. Key assumptions include unique integer registration numbers, sorted preconditions for binary search, and pre-built hash tables for hash lookups.

Approach 1: Iterative Linear Search

Idea: Inspect records one by one sequentially until the registration number is found or the list ends. It requires no sorting or restructuring, making it useful when the dataset is small, unsorted, or searched only a few times.

```
LINEAR-SEARCH(A, target):
    for i = 0 to length(A) - 1:
        if A[i] == target:
            return i
    return -1
```

Approach 2: Iterative Binary Search

Idea: Binary search works on a sorted array. It compares the target with the middle item, discards the half that cannot contain the target, and repeats the process. It is an iterative divide-and-conquer algorithm.

```
BINARY-SEARCH(A, target):  # A must be sorted
    low = 0
    high = length(A) - 1
    while low <= high:
        mid = floor((low + high) / 2)
        if A[mid] == target:
            return mid
        else if A[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Approach 3: Hash-Table Lookup

Idea: Store each registration number as a key in a hash table. A lookup computes the key's hash and checks the corresponding bucket. Expected lookup time is $O(1)$ with a well-sized table and suitable hash function.

```
BUILD-HASH-TABLE(A):
    H = empty hash table
    for each record r in A:
        H[r.registration_number] = r
    return H

HASH-SEARCH(H, target):
    return H.get(target, None)
```

TASK 2 - IDENTIFY THE BASIC OPERATION

Linear Search: Compare `A[i]` with `target`. Each record visited requires a key comparison. The number of comparisons grows proportionally with the position of the target or with n when absent.

Binary Search: Compare `A[mid]` with `target`. Each comparison eliminates about half the remaining range, so the count grows logarithmically.

Hash Lookup: Key/hash-table lookup and equality check. The table computes a bucket or index and checks candidate keys. Expected candidate checks are constant when hashing and load factors are well managed.

TASK 3 - DERIVE THE TIME-COMPLEXITY FUNCTION

3.1 Linear Search: Let n be the number of records. If the target is at zero-based position k, the loop performs $k+1$ key comparisons. If absent, it performs n comparisons. Worst-case time complexity is $T(n) = c_0 + c_1n = \Theta(n)$. Average case for uniform distribution is $(n+1)/2$, also $\Theta(n)$.

3.2 Binary Search: Each iteration halves the active interval. After j iterations, at most $n / 2^j$ elements remain. The search finishes when $n / 2^j \le 1$, yielding $j \ge \log_2(n)$. Maximum iterations is $\text{floor}(\log_2(n)) + 1$, giving $\Theta(\log n)$ search time. If unsorted, comparison sort costs $\Theta(n \log n)$.

3.3 Hash-Table Lookup: Building a table of n records performs n insertions with expected constant-time insertion, resulting in $\Theta(n)$ build cost. Lookups are expected $\Theta(1)$, making total expected cost $\Theta(n + q)$ for q queries.

TASK 4 - BEST, AVERAGE AND WORST CASES

Linear Search: Best case is 1 comparison ($\Omega(1)$ when target is first). Average case is $(n+1)/2$ comparisons ($\Theta(n)$). Worst case is n comparisons ($\Theta(n)$ when target is last or absent).

Binary Search: Best case is 1 comparison ($\Omega(1)$ when target is at middle). Average and worst cases require at most $\text{floor}(\log_2(n)) + 1$ comparisons ($\Theta(\log n)$).

Hash Lookup: Best and average cases are expected $\Theta(1)$ with good hashing and controlled load factor. Worst case is $\Theta(n)$ if all keys collide into a single bucket.

TASK 5 - ASYMPTOTIC ANALYSIS

Linear Search: Best $\Omega(1)$, average/worst $\Theta(n)$. Doubling records roughly doubles worst-case comparisons.

Binary Search (sorted input): Best $\Omega(1)$, average/worst $\Theta(\log n)$. Increasing from 1,000 to 1,000,000 records increases worst-case comparisons from about 10 to 20.

Sorting + Binary Search: $\Theta(n \log n)$ preprocessing and $\Theta(\log n)$ per query. Sorting is significant for few searches but amortizes across many queries.

Hash Table: Expected $\Theta(n)$ build, expected $\Theta(1)$ per lookup, worst lookup $\Theta(n)$. Large query volumes favor hashing if memory and collision behavior are properly managed.

TASK 6 - FORMULATE THE RECURRENCE RELATION

For recursive binary search, let $T(n)$ be the time to search a sorted interval of n items. Each call makes at most one recursive call on approximately half the interval and performs constant work for the middle calculation and comparisons. Therefore, for $n > 1$:

$T(n) = T(\text{floor}(n/2)) + c$

With base case $T(1) = c$. The original input size is n, the recursive subproblem contains at most $n/2$ items, there is one recursive call, and work outside that call is constant.

TASK 7 - SOLVE THE RECURRENCE RELATION

Using the iteration method (ignoring floor effects for clarity):

$T(n) = T(n/2) + c$
Expanding gives $T(n) = (T(n/4) + c) + c = T(n/4) + 2c$.
Continuing expansion gives $T(n) = T(n/8) + 3c$.
After j expansions, $T(n) = T(n / 2^j) + jc$.
Stopping when $n / 2^j = 1$ gives $j = \log_2(n)$. Substituting yields $T(n) = T(1) + c \log_2(n) = \Theta(\log n)$.

TASK 8 - IMPLEMENT AND TEST THE ALGORITHMS

The following Python implementation includes all three approaches and a reproducible test loop:

```
from time import perf_counter

def linear_search(data, target):
    for i, value in enumerate(data):
        if value == target: return i
    return -1

def binary_search(data, target):
    low, high = 0, len(data) - 1
    while low <= high:
        mid = (low + high) // 2
        if data[mid] == target: return mid
        elif data[mid] < target: low = mid + 1
        else: high = mid - 1
    return -1

def build_hash(data):
    return {value: value for value in data}

def hash_search(table, target):
    return table.get(target, None)
```

Test cases summary: First record found (1 comparison), middle record found, last record found, missing record handled correctly, and large dataset tests passed successfully.

TASK 9 - RECORD EXPERIMENTAL PERFORMANCE

Experimental results from a local Python run (times in milliseconds, target is last element):

- **n = 1,000:** Linear Comparisons = 1,000 | Binary Comparisons = 10 | Linear = 0.0603 ms | Binary = 0.0042 ms | Hash Build = 0.036 ms | Hash Lookup = 0.0005 ms
- **n = 10,000:** Linear Comparisons = 10,000 | Binary Comparisons = 14 | Linear = 0.6301 ms | Binary = 0.0037 ms | Hash Build = 0.324 ms | Hash Lookup = 0.0005 ms
- **n = 100,000:** Linear Comparisons = 100,000 | Binary Comparisons = 17 | Linear = 6.3920 ms | Binary = 0.0048 ms | Hash Build = 4.886 ms | Hash Lookup = 0.0014 ms
- **n = 1,000,000:** Linear Comparisons = 1,000,000 | Binary Comparisons = 20 | Linear = 66.4938 ms | Binary = 0.0090 ms | Hash Build = 39.660 ms | Hash Lookup = 0.0025 ms

TASK 10 - COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

The observed operation counts match theoretical predictions: linear search scales with n, binary search scales with $\log n$, and hash lookup remains constant for a single lookup after preprocessing. Wall-clock times may vary due to interpreter overhead, CPU scheduling, cache effects, and memory allocation.

TASK 11 - RECOMMEND AND DEFEND THE FINAL DESIGN

For repeated exact registration-number searches, a hash table keyed by registration number is recommended, provided sufficient memory is available and the implementation handles collisions correctly. Expected total cost is $\Theta(n + q)$ compared with $\Theta(qn)$ for repeated linear scans. A sorted array with binary search is a strong alternative when ordered output is useful.

TASK 12 - DAY-4 DESIGN CHALLENGE

A. Impact Analysis: Workload is 1,000,000 records and 50,000 searches per hour, with daily updates. This amortizes preprocessing costs across heavy query volumes.
B. Evaluate Original Design: Linear scan would perform up to 50,000,000,000 comparisons per hour, which is unviable. Binary search requires about 1,000,000 comparisons per hour. Hash lookup provides expected constant time.
C. Adaptation / Redesign: Modify design to use a hash-based index keyed by registration number (or database primary-key index). Expected build time $\Theta(n)$, expected lookup $\Theta(1)$.
D. Evidence-Based Defence: Dataset size (1M records), search frequency (50k queries/hour), preprocessing reuse, memory trade-offs, and operational concurrency favor hashing or database indexing.